

Lab Report 1: GCD (Euclidean Algorithm)

Theory

The Greatest Common Divisor (GCD) of two integers is the largest positive integer that divides both numbers without leaving a remainder. The most efficient classical algorithm to compute GCD is the Euclidean Algorithm, based on the principle that $\text{GCD}(a, b) = \text{GCD}(b, a \bmod b)$, repeating until the remainder becomes zero.

The iterative approach avoids recursion overhead, using a simple loop to repeatedly replace (a, b) with $(b, a \% b)$ until b becomes 0, at which point a holds the GCD. This algorithm has deep roots in number theory and is one of the oldest known algorithms, attributed to Euclid (300 BC).

Algorithm (Pseudocode)

```
ALGORITHM GCD_Iterative(a, b)
  Input: Two non-negative integers a and b
  Output: GCD of a and b

  WHILE b != 0 DO
    temp <- b
    b <- a MOD b
    a <- temp
  END WHILE
  RETURN a
```

Python Program

```
def gcd_iterative(a, b):
    while b != 0:
        a, b = b, a % b

    return a

if __name__ == "__main__":
    test_cases = [(48, 18), (100, 75), (56, 98), (0, 5), (17, 13)]

    print(f"{'a':>5} {'b':>5} {'GCD':>5}")
    print("-" * 21)

    for a, b in test_cases:
        print(f"{a:>5} {b:>5} {gcd_iterative(a, b):>5}")
```

Output

a	b	GCD
48	18	6
100	75	25
56	98	14
0	5	5
17	13	1

Complexity Analysis

Time Complexity

1. Best Case: $O(1)$

The best case occurs when the algorithm terminates in a single step i.e., either $b = 0$ or $a \bmod b = 0$

Thus, $B(n) = O(1)$

2. Average Case: $O(\log n)$

Each step replaces $(a, b) \rightarrow (b, a \bmod b)$

The remainder is significantly smaller than the previous number which leads to a logarithmic number of steps relative to the size of the input.

Thus, $A(n) = O(\log n)$

3. Worst Case: $O(\log \min(a, b))$

The worst case occurs when the reduction is as slow as possible. This happens when a and b are consecutive Fibonacci numbers.

Thus, $W(n) = O(\log \min(a, b))$

Space Complexity

Only a few variables used (a, b, r) for the iterative method.

Thus, $S(n) = O(1)$

Conclusion

The iterative GCD algorithm using the Euclidean method is highly efficient. Its logarithmic time complexity $O(\log \min(a, b))$ makes it scalable even for very large integers. The constant $O(1)$ space usage is optimal. It is the standard algorithm used in cryptographic applications (RSA key generation, modular arithmetic) due to its speed and correctness.